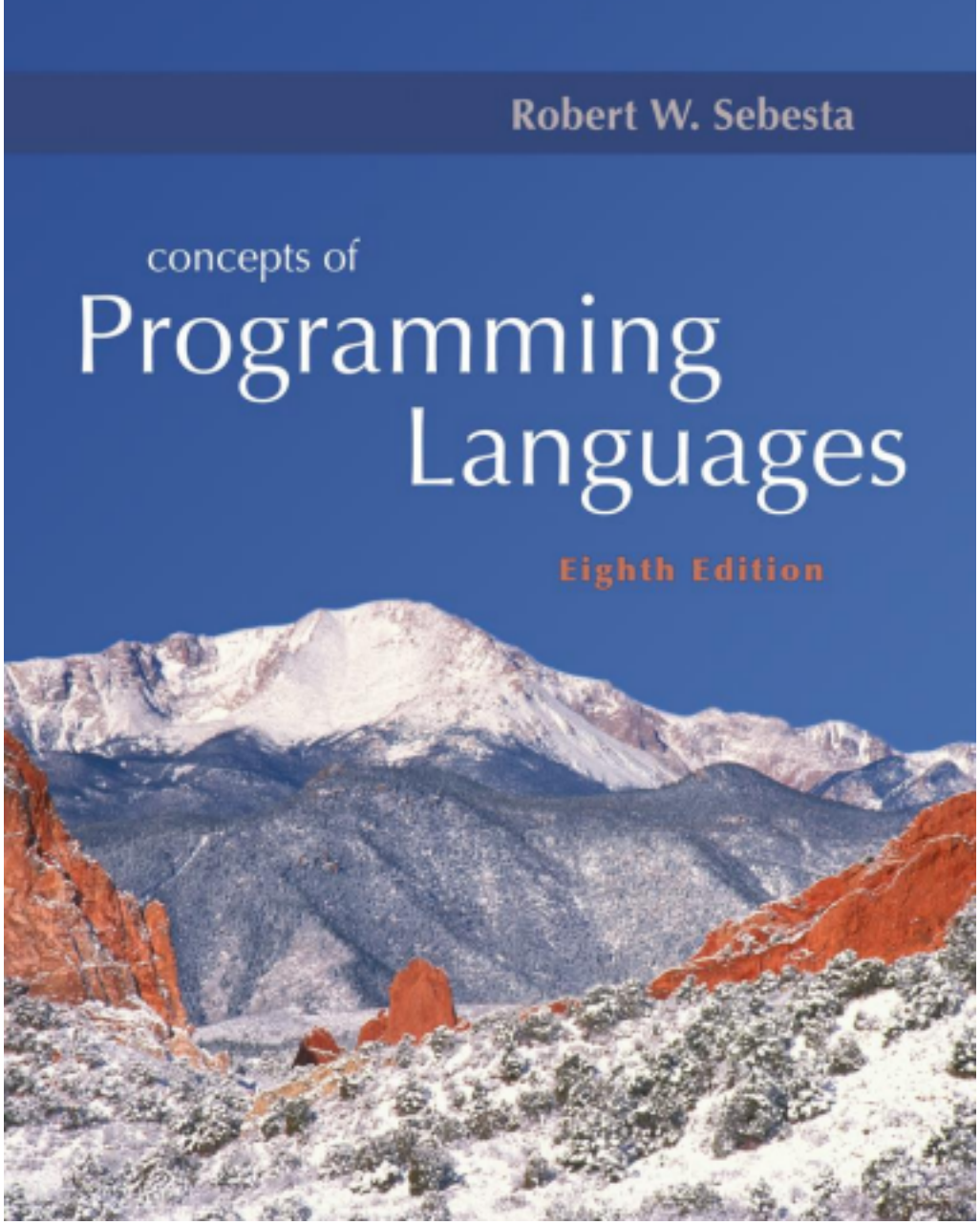
Chapter

6 Data



Types

ISBN 0-321-49362-1

Chapter 6 Topics

- Introduction
- Primitive Data Types

- Character String Types
- User-Defined Ordinal Types
- Array Types
- Associative Arrays
- Record Types
- Union Types
- Pointer and Reference Types

Compile Online Sites

- http://codepad.org
- http://coliru.stacked-crooked.com/

Introduction

- A data type defines a collection of data objects and a set of predefined operations on those objects
- A descriptor is the collection of the attributes of a variable
- An object represents an instance of a

user-defined (abstract data) type

- One design issue for all data types: What operations are defined and how are they specified?

Primitive Data Types

- Almost all programming languages provide a set of primitive data types
- Primitive data types: Those not defined in terms of other data types

- Some primitive data types are merely reflections of the hardware
- Others require little non-hardware support

Primitive Data Types: Integer

- Almost always an exact reflection of the hardware so the mapping is trivial
- There may be as many as eight different

integer types in a language

- Java's signed integer sizes: `byte`, `short`, `int`, `long`

Primitive Data Types: Floating Point

- Model real numbers, but only as approximations

- Languages for scientific use support at least two floating-point types (e.g., `float` and `double`; sometimes more
- Usually exactly like the hardware, but not always
- IEEE Floating-Point Standard 754

Primitive Data Types: Decimal

- For business applications (money) – Essential to COBOL
  - C# offers a decimal data type
- Store a fixed number of decimal digits
- Advantage: accuracy
- Disadvantages: limited range, wastes memory

Primitive Data Types: Boolean

- Simplest of all
- Range of values: two elements, one for "true" and one for "false"

```
int X=3;
if(X%2)
printf("Even");
else
printf("Odd");
```

- Could be implemented as bits, but often as bytes
  - Advantage: readability

Primitive Data Types: Character

- Stored as numeric codings
- Most commonly used coding: ASCII
- An alternative, 16-bit coding: Unicode
  - Includes characters from most natural languages
    - Originally used in Java
    - C# and JavaScript also support Unicode

Character String Types

- Values are sequences of characters
- Design issues:
  - Is it a primitive type or just a special kind of array?
  - Should the length of strings be static or dynamic?

Character String Types Operations

- Typical operations:
  - Assignment and copying
  - Comparison (=, >, etc.)
  - Catenation

- Substring reference
- Pattern matching

Pattern Matching (Regular Expressions)

- A regular expression (regex for short) is a special text string for describing a search pattern
- You can think of regular expressions as

wildcards

- A regular expression is written in a formal language that can be interpreted by a regular expression processor
- such as "*.txt" to find all text files in a file manager
- https://regex101.com

Regular Expressions (Cont')

	Any literal letter, number, or punctuation character (other than those that follow) matches itself.	apple matches apple.
	Patterns can be grouped together using parentheses so that they can be treated as a unit.	see following

	Match a single character (except linefeed).	s.t matches sat, sit, sQt, s3t, s&t, s t,...
	Match zero or one of the previous character/expression. (When immediately following ?, +, *, or {min,max} it prevents the expression from using "greedy" evaluation.)	colou?r matches color, colour
	Match one or more of the previous character/expression.	a+rgh! matches argh!, aargh!, aaargh!,...
	Match zero or more of the previous character/expression.	b(an)*a matches ba, bana, banana, bananana,...
	Match exactly number copies of the previous character/expression.	.o{2}n matches noon, moon, loon,...

Regular Expressions (Cont')

	Match between min and max copies (inclusive) of the previous character/expression.	kabo{2,4}m matches kaboom, kabooom, kaboooom.
	Match a single character in set (list and/or range). Most characters that have special meanings in regular expressions do not have to be backslash-escaped in character sets.	J[aio]b matches Jab, Jib, Job [A-Z][0-9]{3} matches Canadian postal codes.
	Match a single character not in set (list and/or range). Most characters that have special meanings in regular expressions do not have to be backslash-escaped in character sets.	q[^u] matches very few English words (Iraqi? qoph? qintar?).
	Match either expression that it separates.	(Mi|U)nix matches Minix and Unix
	Match the start of a line.	^# matches lines that begin with #.
	Match the end of a line.	^$ matches an empty

		line.

Regular Expressions (Cont')

	Interpret the next metacharacter character literally, or introduce a special escaped combination (see following).	* matches a literal asterisk.
	Match a single newline (carriage return in Python) character.	Hello\nWorld matches Hello World.
	Match a single tab character.	Hello\tWorld matches Hello World.
	Match a single whitespace character.	Hello\s+World matches Hello World, Hello World, Hello World,...
	Match a single non-whitespace character.	\S\S\S matches AAA, The, 5-9,...

	Match a single digit character.	\d\d\d matches 123, 409, 982,...
	Match a single non-digit character.	\D\D matches It, as, &!,...

Regular Expressions in Python

- Before you can use regular expressions in your program, you must import the library using "import re“

Regular Expressions in Python (Cont')

- The re.search() returns a True/False depending on whether the string matches the regular expression

Regular Expressions in Python (Cont')

- If we want the matching strings, we use re.findall()

- ['30', '5', '1999']

Regular Expressions in Python (Cont')

- Finding proper names, starting with a capital letter

- ['Ahmed', 'Ali']

Regular Expressions in Python (Cont’)

- If there is no string matched with regular expression, findall() return an empty list

- []

Regular Expressions in Python (Cont')

- Greedy matching is to match the largest possible string

- ['300$ while Ali found 350$']

Regular Expressions in Python

(Cont') · Non-Greedy matching

- ['300$', '350$']

Regular Expressions in Python

(Cont') · Non-Greedy matching for HTML

- ['<H1>Some text </H1><H1>Some text again </H1>']

Regular Expressions in Python (Cont')

- Parenthesis are not part of the match - but they tell where to start and stop

- ['Some text ', 'Some text again ']

Regular Expressions in Python (Cont')

- Extracting a host name
- You can see this code on https://ideone.com/aF3pDE

- ['gmail.com']

Character String Type in Certain Languages

- C and C++
  - Not primitive
  - Use **`char`** arrays and a library of functions that provide operations
- SNOBOL4 (a string manipulation language) – Primitive
  - Many operations, including elaborate pattern matching

- Java
  - Primitive via the `String` class

Character String Length Options

- Static: COBOL, Java's `String` class
- Limited Dynamic Length: C and C++ – In C-based language, a special character is used to indicate the end of a string's characters, rather than maintaining the length
- Dynamic (no maximum): SNOBOL4, Perl,

JavaScript

- Ada supports all three string length options

Example of String in C Language

```
#include <stdio.h>
#include <stdlib.h>
int main () {
    char * s= "sdfsd\0sfdfsd";
    printf("%s",s);
```

```
    s[5]='3';
    printf("%s",s);
    return 0;
}
```

- http://codepad.org/a0U4fpQw

Character String Type Evaluation

- Aid to writability
- As a primitive type with static length, they are inexpensive to provide--why not have

them?

- Dynamic length is nice, but is it worth the expense?

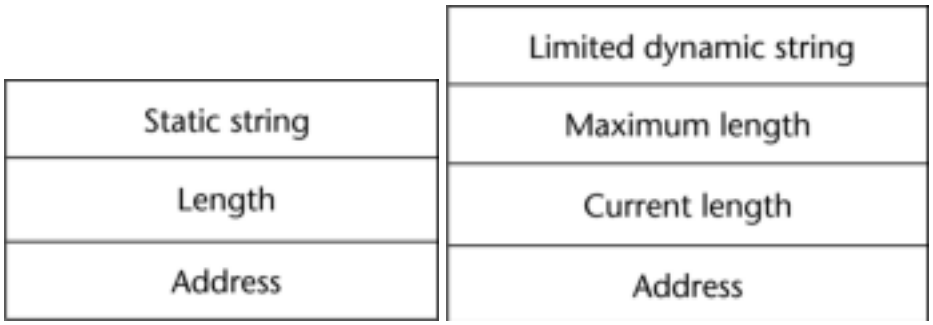
Character String Implementation

- Static length: compile-time descriptor
- Limited dynamic length: may need a run time

descriptor for length (but not in C and C++)

- Dynamic length: need run-time descriptor; allocation/de-allocation is the biggest implementation problem

Compile- and Run-Time Descriptors



Compile-time descriptor for static strings

descriptor for limited dynamic strings

Run-time

User-Defined Ordinal Types

- An ordinal type is one in which the range of possible values can be easily associated with the set of positive integers
- Examples of primitive ordinal types in Java
  - `integer`
  - `char`
  - `boolean`

Enumeration Types

- All possible values, which are named constants, are provided in the definition
- C# example

```
enum days {mon, tue, wed, thu, fri, sat,
sun};
```

- Design issues
  - Is an enumeration constant allowed to appear in

more than one type definition, and if so, how is the type of an occurrence of that constant checked?

- Are enumeration values coerced to integer?

Evaluation of Enumerated Type

- Aid to readability, e.g., no need to code a color as a number
- Aid to reliability, e.g., compiler can check:
  - operations (don't allow colors to be added) –

No enumeration variable can be assigned a value outside its defined range

Example: Enumeration in C++

```
#include <iostream>
int main(){
    enum Color { RED, GREEN, BLUE };
    Color r = RED;
```

```
    switch(r) {
        case RED : std::cout << "red" "\n";
        break;
        case GREEN : std::cout << "green" "\n"; break;
        case BLUE : std::cout << "blue" "\n";
        break;
    }
}
```

- http://codepad.org/eYCzkdcx

Subrange Types

- An ordered contiguous subsequence of an ordinal type

- Example: 12..18 is a subrange of integer type

- Ada's design

```
type Days is (mon, tue, wed, thu, fri, sat,
sun); subtype Weekdays is Days range mon..fri;
subtype Index is Integer range 1..100;

Day1: Days;
Day2: Weekday;
Day2 := Day1;
```


Subrange Evaluation

- Aid to readability
  - Make it clear to the readers that variables of subrange can store only certain range of values
- Reliability
  - Assigning a value to a subrange variable that is outside the specified range is detected as an error

Implementation of User-Defined Ordinal Types

- Enumeration types are implemented as integers
- Subrange types are implemented like the parent types with code inserted (by the compiler) to restrict assignments to subrange variables

Array Types

- An array is an aggregate of homogeneous data elements in which an individual element is identified by its position in the aggregate, relative to the first element.

Array Design Issues

- What types are legal for subscripts? • Are subscripting expressions in element

references range checked?

- When are subscript ranges bound?
- When does allocation take place? •

What is the maximum number of subscripts?

- Can array objects be initialized?
- Are any kind of slices allowed?

Array Indexing

- Indexing (or subscripting) is a mapping

from indices to elements

`array_name (index_value_list) → an element`

- Index Syntax
  - FORTRAN, PL/I, Ada use parentheses
    - Ada explicitly uses parentheses to show uniformity between array references and function calls because both are mappings
  - Most other languages use brackets

Arrays Index (Subscript) Types

- FORTRAN, C: integer only
- Pascal: any ordinal type (integer, Boolean, char, enumeration)
- Ada: integer or enumeration (includes Boolean and char)
- Java: integer types only
- C, C++, Perl, and Fortran do not specify range checking
- Java, ML, C# specify range checking

Subscript Binding and Array Categories

- Static: subscript ranges are statically bound and storage allocation is static (before run-time)
  - Advantage: efficiency (no dynamic allocation)
- Fixed stack-dynamic: subscript ranges are statically bound, but the allocation is done at declaration time
  - Advantage: space efficiency

Subscript Binding and Array Categories (continued)

- Stack-dynamic: subscript ranges are dynamically bound and the storage allocation is dynamic (done at run-time)
  - Advantage: flexibility (the size of an array need not be known until the array is to be used)
- Fixed heap-dynamic: similar to fixed stack dynamic: storage binding is dynamic but

fixed after allocation (i.e., binding is done when requested and storage is allocated from heap, not stack)

Subscript Binding and Array Categories (continued)

- Heap-dynamic: binding of subscript ranges and storage allocation is dynamic and can change any number of times
  - Advantage: flexibility (arrays can grow or shrink during program execution)

Subscript Binding and Array Categories (continued)

- C and C++ arrays that include `static` modifier are static
- C and C++ arrays without `static` modifier

are fixed stack-dynamic

- Ada arrays can be stack-dynamic
- C and C++ provide fixed heap-dynamic arrays
- C# includes a second array class `ArrayList` that provides heap-dynamic
- Perl and JavaScript support heap-dynamic arrays

Array Initialization

- Some language allow initialization at the

time of storage allocation

- C, C++, Java, C# example

```
int list [] = {4, 5, 7, 83}
```

- Character strings in C and C++

```
char name [] = "freddie";
```

- Arrays of strings in C and C++

```
char *names [] = {"Bob", "Jake",
"Joe"];
```

- Java initialization of String objects

```
String[] names = {"Bob", "Jake", "Joe"};
```


Arrays Operations

- APL provides the most powerful array processing operations for vectors and matrixes as well as unary operators (for example, to reverse column elements)
- Ada allows array assignment but also catenation
- Fortran provides elemental operations because they are between pairs of array elements
  - For example, + operator between two arrays results in an array of the sums of the element pairs of the two arrays

Rectangular and Jagged Arrays

- A rectangular array is a multi-dimensioned array in which all of the rows have the same number of elements and all columns have the same number of elements
- A jagged matrix has rows with varying number of elements
  - Possible when multi-dimensioned arrays actually appear as arrays of arrays

Slices

- A slice is some substructure of an array; nothing more than a referencing mechanism
- Slices are only useful in languages that have array operations

Slice Examples

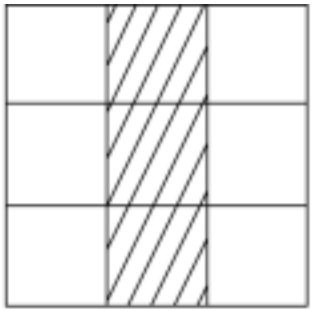
- Fortran 95

```
Integer, Dimension (10) :: Vector
Integer, Dimension (3, 3) :: Mat
Integer, Dimension (3, 3, 4) ::
```

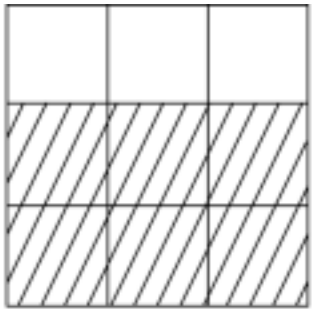
`Cube`

`Vector (3:6)` is a four element array

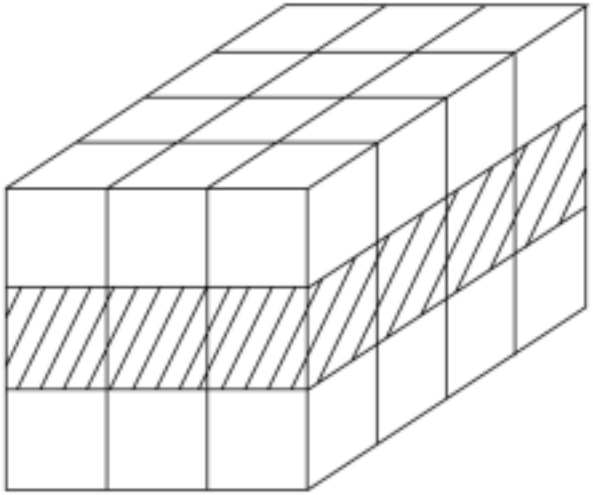
Slices Examples in Fortran 95



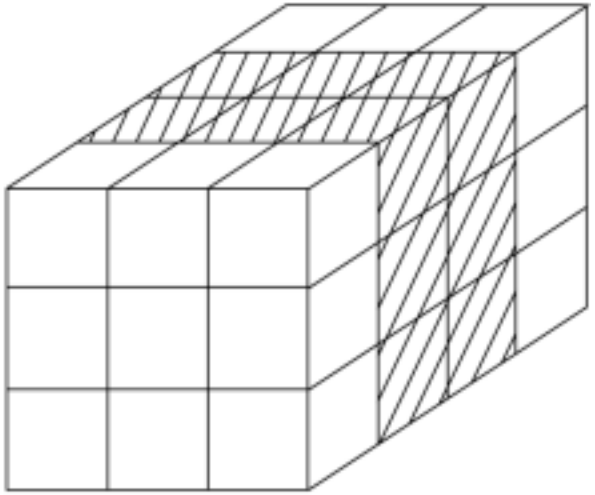
MAT (1:3, 2)



MAT (2:3, 1:3)



CUBE (2, 1:3, 1:4)



CUBE (1:3, 1:3, 2:3)

Implementation of Arrays

- Access function maps subscript expressions to an address in the array
- Access function for single-dimensioned arrays:

  address(list[k]) = address (list[lower_bound])
  + ((k-lower_bound) * element_size)

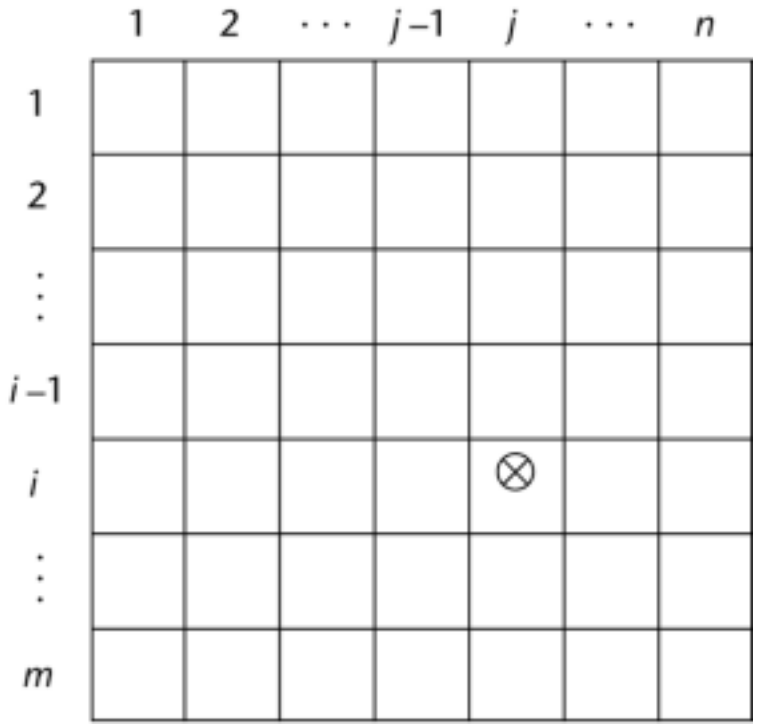
Accessing Multi-dimensioned Arrays

- Two common ways:
  - Row major order (by rows) – used in most languages
  - column major order (by columns) – used in Fortran

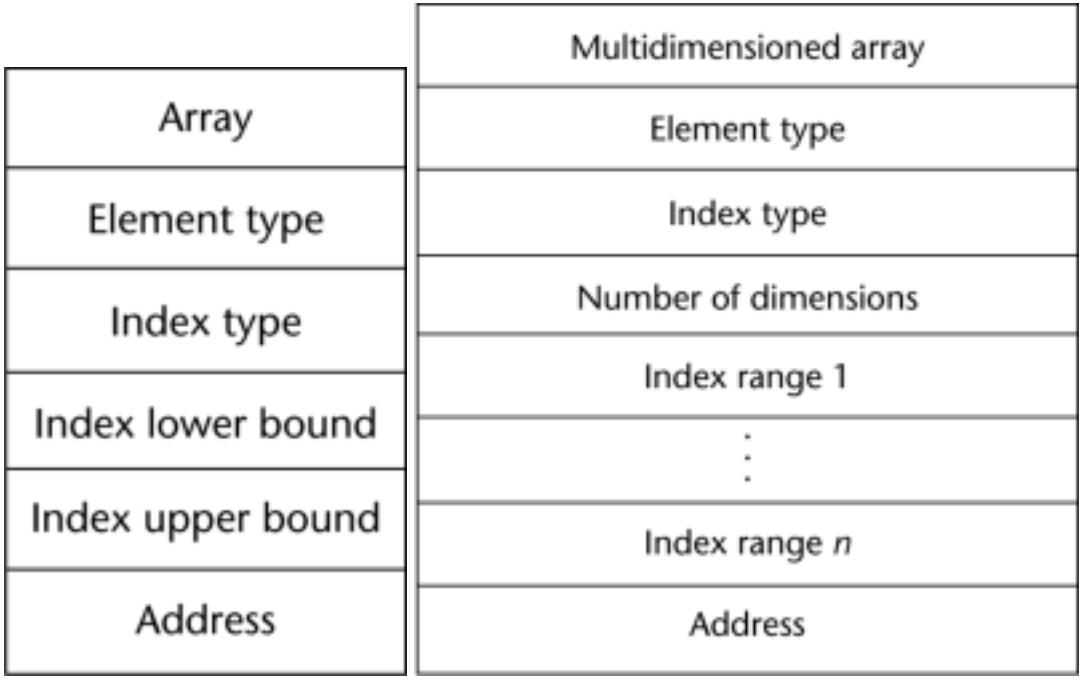
Locating an Element in a Multi dimensioned Array

- General format

  Location (a[I,j]) = address of a [row_lb,col_lb] +
  (((I - row_lb) * n) + (j - col_lb)) * element_size



Compile-Time Descriptors



Single-dimensioned array Multi-dimensional array

Associative Arrays

- An associative array is an unordered collection of data elements that are indexed by an equal number of values called keys
  - User defined keys must be stored
- Design issues: What is the form of references to elements

Associative Arrays in Perl

- Names begin with `%`; literals are delimited by parentheses

```
%hi_temps = ("Mon" => 77, "Tue" => 79,
  "Wed" => 65, …);
```

- Subscripting is done using braces and

keys `$hi_temps{"Wed"} = 83;`

- Elements can be removed with `delete`

```
delete $hi_temps{"Tue"};
```


Associative Arrays in PHP

```
<?php
$age=array("Peter"=>"35","Ben"=>"37","Joe"=>"43");
$age1=array("Peter","Ben",2,9.3);
$age["Ali"] = "343";
```

```
echo "Ali is " . $age['Ali'] . " years old. " . $age1[1] . " " .
$age1[94];
?>
```

https://ideone.com/xtKXeS

Record Types

- A record is a possibly heterogeneous aggregate of data elements in which the individual elements are identified by names

- Design issues:
  - What is the syntactic form of references to the field?
  - Are elliptical references allowed

Definition of Records

- COBOL uses level numbers to show nested

records; others use recursive definition
- Record Field References
  1. COBOL

     field_name OF record_name_1 OF ... OF record_name_n
  2. Others (dot notation)

     record_name_1.record_name_2. ... record_name_n.field_name

Definition of Records in COBOL

- COBOL uses level numbers to show nested records; others use recursive definition

```
01 EMP-REC.
    02 EMP-NAME.
       05 FIRST PIC X(20).
       05 MID PIC X(10).
       05 LAST PIC X(20).
    02 HOURLY-RATE PIC 99V99.
```


Definition of Records in Ada

- Record structures are indicated in an orthogonal way

```
type Emp_Rec_Type is
record First: String
       (1..20);
   Mid: String (1..10);
   Last: String (1..20);
   Hourly_Rate: Float;
end record;
```

```
Emp_Rec: Emp_Rec_Type;
```


References to Records

- Most language use dot notation

  `Emp_Rec.Name`
- Fully qualified references must include all record names
- Elliptical references allow leaving out record names as long as the reference is unambiguous, for example in COBOL `FIRST`, `FIRST OF EMP-NAME`, and `FIRST`

of EMP-REC are elliptical references to the employee's first name

Operations on Records

- Assignment is very common if the types are identical
- Ada allows record comparison
- Ada records can be initialized with aggregate literals

```
type Pair is record
```

```
   x: Integer;
   y: Integer;
end record;
p1: Pair := (5, 6);
```

- COBOL provides `MOVE CORRESPONDING` – Copies a field of the source record to the corresponding field in the target record

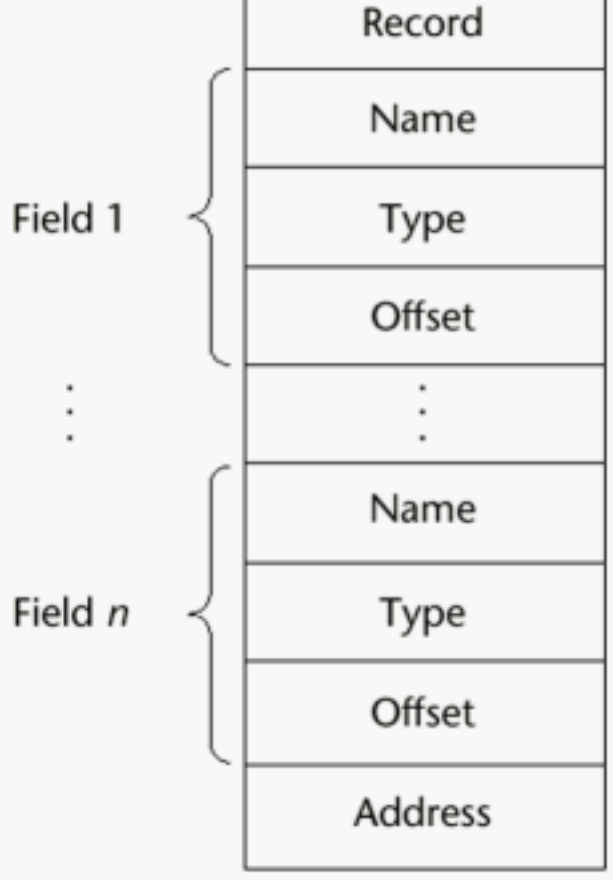
Evaluation and Comparison to Arrays

- Straight forward and safe design
- Records are used when collection of data values is heterogeneous

- Access to array elements is much slower than access to record fields, because subscripts are dynamic (field names are static)
- Dynamic subscripts could be used with record field access, but it would disallow type checking and it would be much slower

Implementation of Record Type

Offset address relative to the beginning of the records
is associated with each field



Unions Types

· A union is a type whose variables are allowed to store different type values at different times during execution · Design issues

- Should type checking be required?
- Should unions be embedded in records?

Discriminated vs. Free Unions

- Fortran, C, and C++ provide union constructs in which there is no language

support for type checking; the union in these languages is called free union

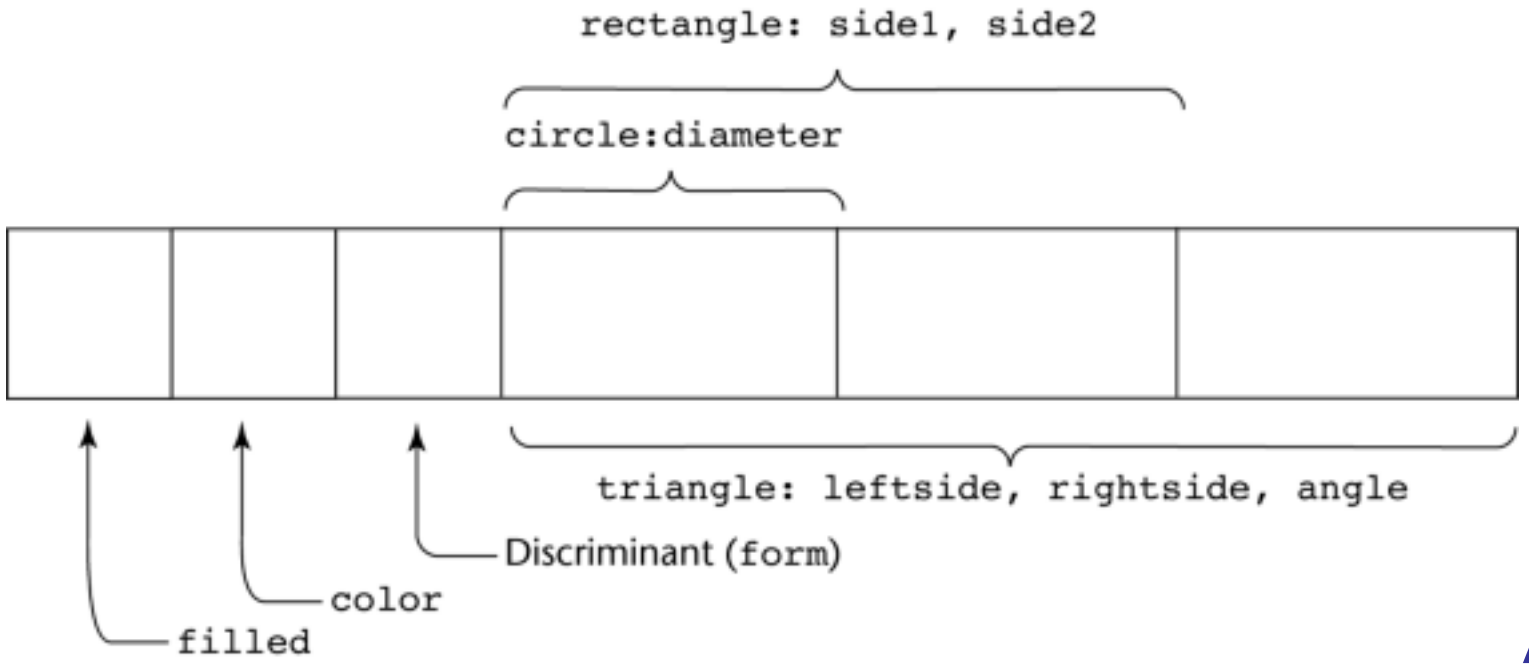
- Type checking of unions require that each union include a type indicator called a discriminant
  - Supported by Ada

Ada Union Types

```
type Shape is (Circle, Triangle,
```

```
Rectangle); type Colors is (Red, Green,
Blue);
type Figure (Form: Shape) is
   record Filled: Boolean;
   Color: Colors;
   case Form is
       when Circle => Diameter: Float;
       when Triangle =>
             Leftside, Rightside: Integer;
             Angle: Float;
       when Rectangle => Side1, Side2:
   Integer; end case;
end record;
```


Ada Union Type Illustrated



A discriminated union of three shape variables

Union in C Language

```
#include "stdio.h"
union data{
struct Bits{
unsigned int x1:1;
unsigned int x2:1;
unsigned int x3:1;
unsigned int x4:1;
}bits;
int y:4;
};
void main(){

union data d1;
d1.y =13;
printf("%d%d%d%d",d1.bits.x4,d1.bits.x3,d1.bits.x2,d1.bits.x1);
char c = getc(stdin);
}
```


Evaluation of Unions

- Potentially unsafe construct
  - Do not allow type checking
- Java and C# do not support unions –
  Reflective of growing concerns for safety in programming language

Pointer and Reference Types

- A pointer type variable has a range of values that consists of memory addresses and a special value, nil

- Provide the power of indirect addressing
- Provide a way to manage dynamic memory
- A pointer can be used to access a location
  - in the area where storage is dynamically created (usually called a heap)

Design Issues of Pointers

- What are the scope of and lifetime of a pointer variable?
- What is the lifetime of a heap-dynamic variable?
- Are pointers restricted as to the type of value to which they can point?
- Are pointers used for dynamic storage management, indirect addressing, or both?
- Should the language support pointer types, reference types, or both?

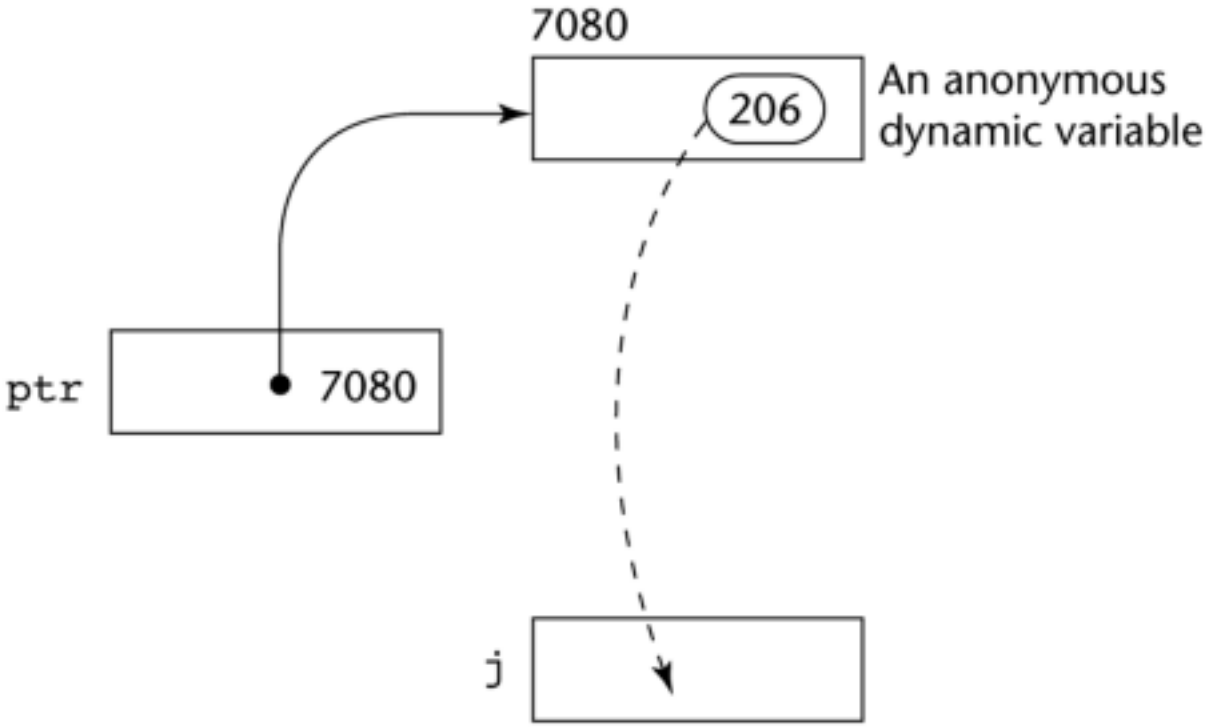
Pointer Operations

- Two fundamental operations: assignment and dereferencing
- Assignment is used to set a pointer variable's value to some useful address
- Dereferencing yields the value stored at the location represented by the pointer's value
  - Dereferencing can be explicit or implicit
  - C++ uses an explicit operation via *

```
j = *ptr
```

sets j to the value located at `ptr`

Pointer Assignment Illustrated



The assignment operation j = *ptr

Problems with Pointers

- Dangling pointers (dangerous)
  - A pointer points to a heap-dynamic variable that has been de-allocated
- Lost heap-dynamic variable
  - An allocated heap-dynamic variable that is no longer accessible to the user program (often called garbage)
    - Pointer `p1` is set to point to a newly created heap dynamic variable
    - Pointer `p1` is later set to point to another newly created heap-dynamic variable

Pointers in C and C++

- Extremely flexible but must be used with care
- Pointers can point at any variable regardless of when it was allocated
- Used for dynamic storage management and addressing
- Pointer arithmetic is possible
- Explicit dereferencing and address-of operators
- Domain type need not be fixed (**`void *`**) • `void *` can point to any type and can be type checked (cannot be de-referenced)